

Capítulo 12. Tira de bits

12.1 Representación de números naturales (enteros positivos)

base 10 decimal	base 2 binario	base 16 hexadecimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	1 0000	10
17	1 0001	11
26	1 1010	1A
31	1 1111	1F
32	10 0000	20
159	1001 1111	9F
160	1010 0000	A0
161	1010 0001	A1
255	1111 1111	FF
256	1 0000 0000	100
257	1 0000 0001	101
2716	1010 1001 1100	A9C

12.1.1 Para pasar de la base 10 a base n

Dado un numero entero num y una base n.

Se hace un bucle hasta que el número de la base 10 no sea inferior a n.

mientras $n > \text{num}$

$\text{num \% } n = \text{digito_de_la_base_n}$

$\text{num} = \text{num} / n$

Leyendo los dígitos del último hasta el primero, se obtiene el numero convertido en base n.

Ejemplo 1.

Dado el numero entero 26 se quiere convertir en un numero binario (base 2).

Primera vuelta

Se busca el resto de 26 dividido por 2. Eso da 0.

Se divide 26 por 2. Eso da 13.

Segunda vuelta

Se busca el resto de 13 dividido por 2. Eso da 1.

Se divide 13 por 2. Eso da 6.

Tercera vuelta

Se busca el resto de 6 dividido por 2. Eso da 0.

Se divide 6 por 2. Eso da 3.

Cuarta vuelta

Se busca el resto de 3 dividido por 2. Eso da 1.

Se divide 3 por 2. Eso da 1.

Quinta vuelta

El número es inferior a la base, nos quedamos con el 1.

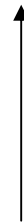
Leyendo del ultimo al primero tenemos 1 1 0 1 0 que si miramos en la tabla es justamente el numero en base 2 de 26 en base 10.

Ejemplo 2.

Esta vez usando una tabla.

Queremos convertir el 35 de la base 10 a la base 2

Numero	Divisor	Cociente	Resto
35	2	17	1
17	2	8	1
8	2	4	0
4	2	2	0
2	2	1	0
1			1



35 en base 10 es igual a 100011 en base 2

Ejemplo 3.

Queremos convertir el 159 de la base 10 a la base 16

Numero	Divisor	Cociente	Resto
159	16	9	F (15)
9			9

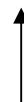


159 en base 10 es igual a 9F en base 16

Ejemplo 4.

Queremos convertir el 2716 de la base 10 a la base 16

Numero	Divisor	Cociente	Resto
2716	16	169	C (12)
169	16	10	9
10			A (10)



2716 en base 10 es igual a A9C en base 16

12.1.2 Para pasar de la base n a base 10

Se multiplica cada digito por su base elevada a la posición del digito en el número. Al final se suman todos los resultados parciales.

Ejemplo 1.

Se quiere convertir el número 100011 de base 2 a base 10.

$$1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 1 \times 2^5 = 1 + 2 + 0 + 0 + 32 = 35$$

Ejemplo 2.

Se quiere convertir el número 10000000 de base 2 a base 10.

$$0 \times 2^0 + 0 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 0 \times 2^5 + 1 \times 2^6 = 0 + 0 + 0 + 0 + 0 + 0 + 64 = 64$$

Ejemplo 3.

Se quiere convertir el número 9F4 de base 16 a base 10.

$$4 \times 16^0 + 15(F) \times 16^1 + 9 \times 16^2 = 4 + 240 + 2304 = 2548$$

Ejemplo 4.

Se quiere convertir el número 1A9B de base 16 a base 10.

$$11(B) \times 16^0 + 9 \times 16^1 + 10(A) \times 16^2 + 1 \times 16^3 = 11 + 144 + 2560 + 4096 = 6811$$

12.1.3 Para pasar de la base 16 a base 2

Cada dígito hexadecimal es equivalente a cuatro dígitos binarios (4 bits). Por lo tanto, los números hexadecimales proporcionan una forma conveniente y concisa para representar números binarios.

Ejemplo 1.

El número binario 10110111 se representa en hexadecimal como B7. Para ver más claramente esta relación se reordenan los bits en grupos de cuatro y cada grupo se representa mediante un dígito hexadecimal; por ejemplo, 1011 0111 representa B 7

Consideramos por ejemplo el número 4E.

Cogemos el primer dígito. El 4 en base 2 es equivalente a 0100.

Cogemos el segundo dígito. La E en base 2 es equivalente a 1110.

Bien el número 4E en base 16 se convierte en 100 1110 en base 2.

Ejemplo 2.

El número 4E90F se convierte en

4 ==> 0100

E ==> 1110

9 ==> 1001

0 ==> 0000

F ==> 1111

Total, el número en base 2 es 0100 1110 1001 0000 1111.

12.1.4 Para pasar de la base 2 a base 16

Se divide el número en grupos de 4 dígitos de base 2 empezando por la derecha y se convierte cada grupo en un dígito en base 16. Si un grupo tiene menos de 4 dígitos, se añaden 0 a la izquierda.

Ejemplo 1.

Queremos convertir el número 1011001 de base 2 a base 16.

Se separa el número en grupos de 4 dígitos, empezando por la derecha del número.

101 ==> 0101 ==> 5

1001 ==> 9

Total, el número 1011001 en base 2 se convierte en 59 en base 16.

Ejemplo 2.

El número 11 0110 1101 0001 1011 en base 2 se convierte en

11 ==> 0011 ==> 3

0110 ==> 6

1101 ==> D

0001 ==> 1

1011 ==> B

El número en base 16 será 36D1B.

12.2 Operaciones a nivel de bits

El C permite realizar operaciones a nivel de bits. Estas operaciones se pueden dividir en tres categorías generales: el operador de complemento a uno, los operadores lógicos y los operadores de desplazamiento.

12.2.1 Asignación de un número hexadecimal

```
unsigned int a, b;
unsigned long c;
unsigned short d;

a = 23356;    // En binario a = 0101 1011 0011 1100
b = 0x5b3c;   // En binario b = 0101 1011 0011 1100, en decimal b = 23356
c = 0xa;      // En binario c = 1010, en decimal c = 10
d = 0xabc;    // En binario d = 1010 1011 1100, en decimal d = 2748

printf( "Inserta un numero hexadecimal: " );
scanf( "%x%c", &a );

printf( "En decimal: %u\n", a );
printf( "En hexadecimal: %x\n", a );
```

12.2.2 Operador de negación (o complemento a uno)

El operador ~ recibe el nombre de operador de negación o complemento a uno. La función de este operador consiste en cambiar los 1 por 0 y viceversa.

Ejemplo 1

```
unsigned short a, b;

a = 0x1b3c;   // En binario a = 0001 1011 0011 1100
b = ~a;       // En binario b = 1110 0100 1100 0011

printf( "En decimal:      a = %u, b = %u\n", a, b );
printf( "En hexadecimal: a = %x, b = %x\n", a, b );
```

Por pantalla saldrá

```
En decimal:      a = 6972, b = 58563
En hexadecimal: a = 1b3c, b = e4c3
```

Ejemplo 2

```
unsigned short a,b;
a = 0x1;       // En binario a = 0000 0000 0000 0001, 4 grupos porque el tipo
                // unsigned short es de 2 bytes (16 bits, 4 grupos de 4 bits)
b = ~a;        // En binario b = 1111 1111 1111 1110

printf( "En decimal:      a = %u, b = %u\n", a, b );
printf( "En hexadecimal: a = %x, b = %x\n", a, b );
```

Por pantalla saldrá

En decimal: a = 1, b = 65534

En hexadecimal: a = 1, b = fffe

Ejemplo 3

```
unsigned int a,b;
```

```
a = 0x4f9; // En binario a = 0000 0000 0000 0000 0000 0100 1111 1001
```

```
b = ~a; // En binario b = 1111 1111 1111 1111 1111 1011 0000 0110
```

```
printf( "En decimal: a = %u, b = %u\n", a, b );
```

```
printf( "En hexadecimal: a = %x, b = %x\n", a, b );
```

Por pantalla saldrá

En decimal: a = 1273, b = 4294966022

En hexadecimal: a = 4f9, b = ffffffb06

12.2.3 Operadores lógicos

Hay tres operadores lógicos a nivel de bits:

- y (& o AND); una operación y a nivel de bits devuelve 1 si ambos bits tienen el valor 1. En otro caso devuelve siempre 0
- o (| o OR); una operación o a nivel de bits devuelve 0 si ambos bits tienen el valor 0. En otro caso devuelve siempre 1.
- o exclusiva (^ o XOR); una operación o exclusiva a nivel de bits devuelve 1 si los dos bits son distintos. Devuelve 0 si ambos bits tienen el mismo valor.

Estos resultados están resumidos en la tabla siguiente.

Bit1	Bit2	AND (&)	OR ()	XOR (^)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Ejemplo 1.

```
unsigned short a, b, c;
```

```
a = 0x6db7; // en binario 0110 1101 1011 0111, en decimal 28087
```

```
b = 0xa726; // en binario 1010 0111 0010 0110
```

```
c = a & b; // AND logico
```

Esto equivale en hacer

```
a = 0110 1101 1011 0111
```

```
b = 1010 0111 0010 0110
```

```
-----
```

```
a & b = 0010 0101 0010 0110
```

```
=      2      5      2      6 // en hexadecimal
```

Ejemplo 2.

```
c = a | b;      // OR logico
```

Esto equivale en hacer

```
    a = 0110 1101 1011 0111
    b = 1010 0111 0010 0110
    -----
a | b = 1110 1111 1011 0111
      =   e   f   b   7      // en hexadecimal
```

Ejemplo 3.

```
c = a ^ b;      // XOR logico (o exclusivo)
```

Esto equivale en hacer

```
    a = 0110 1101 1011 0111
    b = 1010 0111 0010 0110
    -----
a ^ b = 1100 1010 1001 0001
      =   c   a   9   1      // en hexadecimal
```

Enmascaramiento

El enmascaramiento es un proceso en el que un número binario se convierte en otro número por medio de una operación lógica a nivel de bits. El número binario original es uno de los dos operandos en la operación. El segundo operando, llamado *mascara*, es un número binario especialmente escogido que realiza la transformación deseada.

Hay varios tipos distintos de operaciones de enmascaramiento. Por ejemplo, una parte del número binario puede copiarse a un nuevo número rellenando el resto con ceros o unos dependiendo del caso. Así, una parte del número binario original será enmascarado del resultado final.

Ejemplo 1

Supongamos que a es una variable entera sin signo cuyo valor es 0x6db7 (28087 en decimal). Queremos extraer los 6 bits más a la derecha de este valor y los asignaremos a la variable entera sin signo b. Asignaremos ceros a los 10 bits de b más a la izquierda.

```
unsigned short a, b, masc;
```

```
a = 0x6db7;      // en binario 0110 1101 1011 0111
```

Hemos dicho que queremos solo los 6 bits más a la derecha. Ponemos entonces en la máscara masc este valor.

```
masc = 0x3f;      // en binario 0000 0000 0011 1111, 63 en decimal
b = a & masc;
```

La validez de este resultado puede establecerse examinando esta operación

```
    a = 0110 1101 1011 0111
    masc = 0000 0000 0011 1111
    -----
a & masc = 0000 0000 0011 0111
          =   0   0   3   7      // en hexadecimal
```

Notar que la máscara impide que los 10 bits más a la izquierda se copien de a a b.

Ejemplo 2

Supongamos que a es una variable entera sin signo cuyo valor es 0x6db7. Esta vez queremos extraer los 10 bits más a la izquierda de este valor y los asignaremos a la variable entera sin signo b. Asignaremos ceros a los 6 bits de b más a la derecha.

```
a = 0x6db7;    // en binario 0110 1101 1011 0111
```

Hemos dicho que queremos solo los 10 bits más a la izquierda. Ponemos entonces en la máscara masc este valor.

```
masc = 0xffc0;    // en binario 1111 1111 1100 0000
b = a & masc;
```

La validez de este resultado puede establecerse examinando esta operación

```
      a = 0110 1101 1011 0111
      masc = 1111 1111 1100 0000
      -----
a & masc = 0110 1101 1000 0000
          =   6   d   8   0    // en hexadecimal
```

La máscara bloquea ahora los 6 bits más a la derecha de a.

En el caso anterior, la máscara depende del tamaño del tipo de variable usada como número binario (16 bits) ya que los unos aparecen en las posiciones de más a la izquierda. Por esa razón es más recomendable usar una máscara que contenga los 1 más a la derecha y luego hacer la operación de negación para reconducirse al caso anterior. Es decir

En lugar de usar la máscara

```
masc = 0xffc0;    // en binario 1111 1111 1100 0000
```

Es más recomendable usar la máscara

```
masc = 0x3f;      // en binario 0000 0000 0011 1111
```

Y luego hacer

```
b = a & ~masc;
```

La validez de este resultado puede establecerse examinando esta operación

```
      a = 0110 1101 1011 0111
      ~masc = 1111 1111 1100 0000    // masc = 0000 0000 0011 1111
      -----
a & ~masc = 0110 1101 1000 0000
          =   6   d   8   0    // en hexadecimal
```


Ejemplo 3

Supongamos que a es una variable entera sin signo cuyo valor es 0x6db7. Queremos mantener los 8 bits del medio y asignar ceros a los dos grupos de 4 bits de la derecha y de la izquierda. El resultado lo pondremos en b.

```
a = 0x6db7;    // en binario 0110 1101 1011 0111
```

Hemos dicho que queremos los 8 bits del medio. Ponemos entonces en la máscara masc este valor.

```
masc = 0xff0;    // en binario 0000 1111 1111 0000
b = a & masc;
```

La validez de este resultado puede establecerse examinando esta operación

```
      a = 0110 1101 1011 0111
      masc = 0000 1111 1111 0000
      -----
a & masc = 0000 1101 1011 0000
          =   0   d   b   0    // en hexadecimal
```

Notar que la máscara impide que los 4 bits de la derecha y los 4 bits de la izquierda se copien de a a b.

Ejemplo 4

Supongamos que a es una variable entera sin signo cuyo valor es 0x6db7. Esta vez queremos transformar el número binario en otro en el cual los 5 bits más a la derecha sean unos y los 11 bits más a la izquierda conserven su valor original. Asignaremos este nuevo número binario a la variable entera sin signo b.

```
a = 0x6db7;    // en binario 0110 1101 1011 0111
```

Hemos dicho que queremos transformar los 5 bits más a la derecha en unos y conservar los restantes 11 bits. Ponemos entonces en la máscara masc este valor.

```
masc = 0x001f;    // en binario 0000 0000 0001 1111;
b = a | masc;
```

La validez de este resultado puede establecerse examinando esta operación

```
      a = 0110 1101 1011 0111
      masc = 0000 0000 0001 1111
      -----
a | masc = 0110 1101 1011 1111
          =   6   d   b   f    // en hexadecimal
```

Recordar que la operación a nivel de bits es ahora la o (OR). Así, cuando cada uno de los 5 bits de la derecha en a se compara con el correspondiente 1 de la máscara, el resultado es siempre 1. Sin embargo, cuando cada uno de los 11 bits de la izquierda en a se compara con el correspondiente 0 en la máscara, el resultado será el mismo que el bit original en a.

Ejemplo 5

Dado el mismo numero de antes $a = 0x6db7$ queremos transformar este numero en otro en el cual los 11 bits más a la izquierda sean todos unos y los 5 bits de más a la derecha conserven su valor original. Esto se puede hacer mediante cualquiera de las siguientes dos expresiones:

```
b = a | 0xffe0;    // máscara 1111 1111 1110 0000
b = a | ~0x001f;   // máscara 0000 0000 0001 1111
```

Se puede copiar una parte de un número binario a un nuevo número, mientras que el resto del número binario original se invierte dentro del nuevo numero. Este tipo de enmascaramiento utiliza la o exclusiva (XOR).

Ejemplo 6

Supongamos que a es una variable entera sin signo cuyo valor es $0x6db7$. Queremos invertir los 8 bits de más a la derecha y preservamos los 8 bits de más a la izquierda. Asignaremos este nuevo número binario a la variable entera sin signo b .

```
a = 0x6db7;        // en binario 0110 1101 1011 0111
masc = 0x00ff;     // en binario 0000 0000 1111 1111;
b = a ^ masc;
```

La validez de este resultado puede establecerse examinando esta operación

```
      a = 0110 1101 1011 0111
      masc = 0000 0000 1111 1111
      -----
a ^ masc = 0110 1101 0100 1000
           =   6   d   4   8   // en hexadecimal
```

Recordar que la operación a nivel de bits es ahora la o exclusiva (XOR). Por tanto, cuando uno de los 8 bits de la derecha en a se compara con el correspondiente 1 de la máscara, el bit resultante será el opuesto al bit original de a . Por otra parte, cuando uno de los 8 bits de la izquierda en a se compara con el correspondiente 0 en la máscara, el bit resultante será el mismo que el bit original en a .

En resumen, con las mascaras y los operadores podemos:

- y (&) => dejar pasar bits determinados;
- o (|) => poner 1's en posiciones determinadas;
- o exclusivo (^) => invertir los bits de posiciones determinadas.
-

12.2.4 Operadores de desplazamiento de bits

Hay dos operadores de desplazamiento de bits:

- << desplazamiento a la izquierda
- >> desplazamiento a la derecha

Cada operador requiere dos operandos. El primero es un operando de tipo entero que representa el número binario a desplazar. El segundo es un entero sin signo que indica el número de

desplazamientos. Este valor no puede exceder del número de bits asociado con el tamaño del primer operando.

El operador << hace que los bits en el primer operando sean desplazados a la izquierda el número de posiciones indicado por el segundo operando. Se perderán los bits de más a la izquierda en el número binario original. Las posiciones de la derecha que quedan vacantes se rellenan con ceros.

Ejemplo 1.

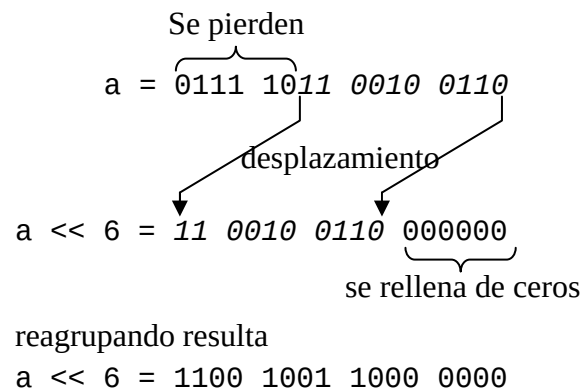
Supongamos que a es una variable entera sin signo cuyo valor es 31526 (0x7b26). La expresión

```
unsigned short b, a = 31526; // equivalente a 0x7b26 en hexadecimal
```

```
b = a << 6;
```

desplazará todos los bits seis posiciones a la izquierda y asignará el valor resultante a la variable entera sin signo b. El valor resultante de b será 51584 (0xc980)

Verificamos este resultado



Todos los bits asignados a a se desplazan seis posiciones a la izquierda como indican las flechas. Los 6 bits de la izquierda (originalmente 0111 10) se pierden. Las seis posiciones de más a la derecha se llenan con 00 0000.

El operador >> hace que los bits en el primer operando sean desplazados a la derecha el número de posiciones indicado por el segundo operando. Se perderán los bits de más a la derecha en el número binario original. Si el número binario que se desplaza representa un entero **sin signo**, las posiciones de la izquierda que quedan vacantes se rellenan con ceros. Por tanto, el comportamiento del operador desplazamiento a la derecha es similar al de desplazamiento a la izquierda cuando el primer operando es un entero sin signo.

Ejemplo 2.

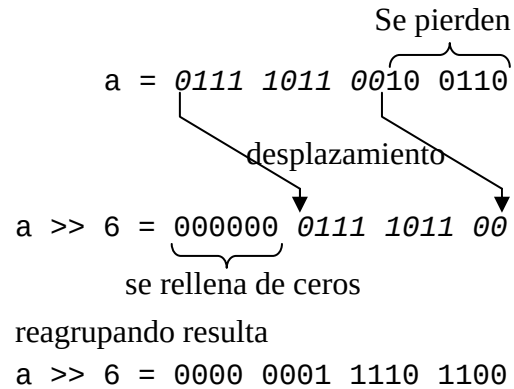
Supongamos que a es una variable entera sin signo cuyo valor es 31526 (0x7b26). La expresión

```
unsigned short b, a = 31526; // equivalente a 0x7b26 en hexadecimal
```

```
b = a >> 6;
```

desplazará todos los bits seis posiciones a la derecha y asignará el valor resultante a la variable entera sin signo b. El valor resultante de b será 492 (0x1ec)

Verificamos este resultado



Todos los bits asignados a a se desplazan seis posiciones a la derecha como indican las flechas. Los 6 bits de la derecha (originalmente 10 0110) se pierden. Las seis posiciones de más a la izquierda se llenan con 0000 00.

Es importante señalar que los operadores de desplazamiento tienen un significado aritmético en base 10. El desplazamiento a la derecha de 1 bit es equivalente a dividir por 2, obteniéndose el cociente de la división entera. El desplazamiento a la izquierda de 1 bit es equivalente a multiplicar por 2. Por lo tanto, cuando trabajemos con variables entera:

variable << n equivale a variable $\times 2^n$

variable >> n equivale a variable $/ 2^n$

Ejercicio 1.

Distancia de Hamming

Se define la distancia de Hamming entre dos tiras de bit, como la cantidad de posiciones en las que las dos tiras de bits difieren. Veamos un ejemplo, supongamos dos tiras de bits de 8 bits (char) como la siguientes:

Tira 1: 0 1 1 0 1 0 1 0

Tira 2: 0 0 1 1 0 1 1 1

Difieren en: X X X X X

Dado que difieren en 5 posiciones, su distancia de Hamming es 5.

Se pide realizar una función que acepte dos enteros (tipo int codificado en 32 bits) y que devuelve la distancia de Hamming entre dicho enteros. La cabecera de la función es la siguiente:

```
int DistanciaHamming( int a, int b )
```

Solución

```
int DistanciaHamming( int a, int b )
{
    int c, m, i, distancia;

    c = a ^ b;      /* o exclusivo, en c se pone un 1 en correspondencia
                     de dos bits distintos */
    m = 1;          /* máscara que irá desplazando este uno y servirá
                     para comprobar si hay bits en c iguales a 1 */
    distancia = 0;   // distancia de Hamming

    for( i = 0; i < 32; i++ )    // bucle de 32 bits
    {
        if( (c & m) != 0 )      /* si el bit que deja pasar m es igual a 1
                                   quiere decir que hemos encontrados un 1 en
                                   c que corresponde a dos bits diferentes
                                   entre a y b */

            distancia++;
        m = m << 1;             /* se desplaza el 1 de una posición a la
                                   izquierda */
    }
    return distancia;
}
```

Ejercicio 2.

La manera habitual de representar números enters és el format complement a 2 (c2) que es pot resumir de la manera següent: si un número és positiu el representem en binari natural, i si és negatiu primer li apliquem l'operació complement a 1 (intercanviar el 1s per 0s i el 0s per 1s), i després sumem 1 al resultat.

Un codi alternatiu és el format signe i magnitud (sm). El bit de major pes indicarà el signe (0 positiu; 1 negatiu) i la resta de bits representen la magnitud (valor absolut) en binari natural.

Un exemple amb paraules de 8 bits:

Valor decimal	Signe i magnitud	Complement a 2
24	0001 1000	0001 1000
-24	1001 1000	1110 1000

Escriu la següent funció per passar un valor de 32 bits del format sm al format c2.

```
unsigned int smAc2( unsigned int num )
```

NOTA: Només podeu usar variables del tipus unsigned int.

```
unsigned int smAc2( unsigned int num )
{
    unsigned int res;

    if( (num & 0x80000000) != 0 )
    {
        res = num & 0x7fffffff;
        res = ~res;
        res = res + 1;
    }
    return res;
}
```

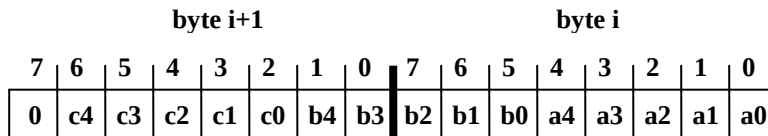
Ejercicio 3.

Debes hacer una función que comprima un texto, y cuya cabecera es la siguiente:

```
void comprime( char entrada[], char salida[], char traduccion[32] )
```

La compresión de un texto corriente es posible porque normalmente lo único que aparece en el texto son letras y pocos símbolos más, en vez de los 256 caracteres posibles que podrían aparecer en cada uno de los bytes (char) que componen un texto. En el texto de entrada (parámetro entrada) se garantiza que como máximo pueden aparecer 32 caracteres diferentes. Por lo tanto, a cada uno de estos caracteres le corresponde un número de 5 bits, que indica la posición que ocupa dentro de un vector de traducción (parámetro traducción).

La función comprime deberá traducir el vector de entrada a uno de salida (parámetro salida) que contenga 3 letras comprimidas cada 2 bytes. Para hacer la compresión, se deben ir traduciendo 3 caracteres consecutivos del vector de entrada y guardar las traducciones en 3 variables temporales llamadas a, b y c. A continuación, se deben empaquetar estas 3 variables en 2 bytes (16 bits) del vector de salida. El bit extra que sobra se pondrá a 0. En el diagrama inferior se muestra que posiciones han de ocupar cada uno de los bits significativos de las variables a, b y c en el vector de salida.



El algoritmo acaba cuando en el vector de entrada aparezca un ‘\0’. Este ‘\0’ es el último carácter que se debe traducir. Puedes suponer que en el vector de salida siempre habrá espacio para hacer la traducción de todo el vector de entrada. También puedes suponer que existe una función que dado un carácter y el vector de traducción te devuelve la posición dentro del vector de dicho carácter. La cabecera de esta función es:

```
int traduce( char traduccion[32], char car )
```

Solución

Función ya dada: `int traduce( char traduccion[32], char car);`

```
void comprime( char entrada[], int salida[], char traduccion[32] )
{
    char   x, y, z;
    int    i, j;

    i = 0;
    j = 0;
    do
    {
        x = entrada[i];          // se coge el primer carácter
        i++;
        y = traduce( traduccion, x ); // se traduce el carácter
        y = y & 0x1F;             /* se eliminan los 3 bits que sobran del
                                   carácter traducido aplicando una máscara
                                   del tipo 0001 1111 */
    } while ( i < strlen(entrada) );
}
```

```

salida[j] = y;          /* se guarda el carácter traducido en la cadena
                        de salida */

if (x != '\0')         // si no hemos llegado al ultimo seguimos
{
    x = entrada[i];     // se coge otro carácter
    i++;
    y = traduce( traduccion, x ); // se traduce
    y = y & 0x1F;        /* se aplica la misma máscara para eliminar los
                        tres bits mas a la izquierda */
    z = y;              // se guarda y en la z
    y = y << 5;          /* se desplaza la y de 5 posiciones a la izquierda
                        eso porque tenemos que poner solo tres bits
                        en los tres bits que quedan vacíos en la cadena
                        de salida. En el ejemplo de la figura estamos
                        construyendo los bits b0, b1 y b2. */

    salida[j] = salida[j] | y; /* añadido estos tres bits a la cadena de
                        salida en los tres bits que quedan */
    j++;                  /* la primera cadena se ha acabado, pasamos a la
                        segunda cadena */
    y = z >> 3;           /* desplazamos la z (que era la y original) de 3
                        hacia la derecha para eliminar los 3 bits que
                        ya hemos puesto anteriormente. Así nos quedaran
                        solo los bits b4 y b3 */
    salida[j] = y;        // ponemos estos bits en la segunda cadena

    if (x != '\0')       // si no hemos llegado al ultimo seguimos
    {
        x = entrada[i]; // se coge otro carácter
        i++;
        y = traduce( traduccion, x ); // se traduce
        y = y & 0x1F;    /* se aplica la misma máscara para eliminar los
                        tres bits mas a la izquierda */

        y = y << 2;      /* se deplaza de 2 posiciones a la izquierda
                        para crear los carácter c4, c3, c2, c1 y c0
        salida[j] = salida[j] | y; /* se añaden estos caracteres a la
                        segunda cadena */
        j++;
    }
}
}while (x != '\0');      /* se han acabados los 2 bytes y si quedan
                        otros caracteres seguimos adelante con otras
                        dos cadenas */
}

```